

1 Sets, Relations and Languages

1.1 Sets

- Set notion

1. Enumeration method For example, $A = 1, 2, 3$, or $N = 0, 1, 2, \dots$
2. Descriptive method For example, the set of odd natural numbers is $O = \{x : x \in N \text{ and } x \text{ is not divisible by } 2\}$.
3. Singleton Singleton set contains only one element.
4. Empty Set ϕ
5. 集合中的元素之间可以没有任何关系
6. Sets have Reciprocity (互异性), $\{1, 2, 3\}$ and $\{1, 1, 2, 3\}$ 是相同的集合。
7. The set does not distinguish the order of the elements. So, two sets are the same if and only if they have the same elements.

- 子集和真子集

If every element of set A is an element of set B, then A is said to be a subset of B, $A \subseteq B$.

If A is a subset of B but A is not the same as B, so A is a proper subset of B and write $A \subset B$.

- Set operations

1. Intersection $A \cap B = \{x, x \in A \text{ and } x \in B\}$.
 $A = \{1, 3, 9\}$, $B = \{3, 5, 7\}$, $A \cap B = \{3\}$
2. Union $A \cup B = \{x, x \in A \text{ or } x \in B\}$
3. The difference of two sets $A - B = \{x, x \in A \text{ and } x \notin B\}$
4. Idempotency 幂等律: $A \cap A = A$, $A \cup A = A$
5. Commutativity 交换律: $A \cup B = B \cup A$, $A \cap B = B \cap A$
6. Associativity 结合律: $(A \cup B) \cup C = A \cup (B \cup C)$, $(A \cap B) \cap C = A \cap (B \cap C)$

7. Distributivity 分配律: $(A \cup B) \cap C = (A \cap C) \cup (B \cap C)$, $(A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$

8. Absorption 吸收率: $(A \cap B) \cup A = A$, $(A \cup B) \cap A = A$

9. DeMorgan's laws 德摩根定律: $A - (B \cap C) = (A - B) \cup (A - C)$,
 $A - (B \cup C) = (A - B) \cap (A - C)$

- 证明 DeMorgan's laws 德摩根定律: $A - (B \cap C) = (A - B) \cup (A - C)$

Using the theorem, two sets are the same if and only if they have the same elements. We must prove $L \subseteq R$ and $R \subseteq L$. Let $L = A - (B \cap C)$, $R = (A - B) \cup (A - C)$.

Firstly, let x be any element of L ; then $x \in A$, but $x \notin B$ and $x \notin C$. Hence x is an element of $A - B$ and $A - C$, and x is thus an element of R , therefore $L \subseteq R$.

Secondly, let x be any element of R ; then x is both in $x \in (A - B)$ and $x \in (A - C)$, and x is therefore in A but in neither B or C . Hence $x \in A$ but $x \notin B \cup C$, so $x \in L$. Therefore $R \subseteq L$.

- binary operation of the Set

$$\cup S = \{x : x \in P \text{ for some set } P \in S\}$$

$$\cap S = \{x : x \in P \text{ for each set } P \in S\}$$

For example, $S = \{\{a, b\}, \{b, c\}, \{c, d\}\}$, $\{a, b\} \cup \{b, c\} \cup \{c, d\} = \{a, b, c, d\} = \cup S$,

- A 的幂集

The power set of A , denoted 2^A , is the collection of all subsets of a set of A .

eg: $2^{\{c, d\}} = \{\{c, d\}, \{c\}, \{d\}, \phi\}$.

- Partition of a Set

一个非空集合 A 的 Partition 是其幂集 2^A 的子集 Π , 并且在 Π 中不包括 ϕ , 每一个 A 中的元素都只在 Π 中一个集合。

Π is a partition of A if Π is a set of subset of A , such that

(1) each element of Π is nonempty

(2) distinct members of $\mathcal{H}$ are disjoint

(3) $\cup \mathcal{H}$ is A

1.2 Relations and Functions

- 有序对

Ordered pairs represent the relationship between objects. In the ordered pair (a,b) , a and b are the components, and (a,b) is not the same as (b,a) .

- Cartesian product(笛卡尔积)

The Cartesian product of two sets A and B , denoted by $A \times B$, is the set of ordered pairs (a,b) with $a \in A$ and $b \in B$.

两集合上的二元关系 (denoted by Q), 就是它们笛卡尔积的任意子集。A binary relation on two sets A and B are sets of ordered pairs (a, b) of any $a \in A$ and $b \in B$.

- n -ary relation(n 元关系)

如果 $A_1, A_2, \dots, A_n$ 是任一集合, 这些集合的 n 折笛卡尔积是所有有序 n 元组的集合 $(a_1, a_2, \dots, a_n)$.

An ordered m -tuple $(b_1, \dots, b_m)$ is the same as $(a_1, \dots, a_n)$ if and only if $m=n$ and $a_i=b_i$, for $i=1, \dots, n$

- Functions

1. 定义: A function is an association of each object of one kind with unique object of another kind.
2. $f : A \rightarrow B$ indicate f is the function from A to B , where A is the domain of f and B is the range of f . If $a \in A$, $f(a)$ is the image(像) of a under f .
3. 从集合 A 到集合 B 的函数是一个在 A 和 B 上的关系 R , for each element $a \in A$ there is an exactly one ordered pair in R with first component a .
4. 单射, $f : A \rightarrow B$ is one-to-one if for any two distinct elements $a, a' \in A, f(a) \neq f(a')$.

5. 满射, $f : A \rightarrow B$ is onto B, if each element of B is the image under f of some elements of A.
6. 双射, $f : A \rightarrow B$ is bijection between A and B if both one-to-one and onto B.

1.3 Special types of Binary Relations

Node can be represented by the element of set, arrow is the relation direction, then node and arrow can construct directed/undirected graph.

- 自反关系

A relation $R \subseteq A * A$ is reflexive if $(a, a) \in R$ and $a \in A$. 在有向图, 一个自反关系表示每个节点都有一条指向自身的环。

- 对称关系

A relation $R \subseteq A * A$ is symmetric if $(a, b) \in R$ whenever $(b, a) \in R$. 在有向图中表示两个节点一定有双向箭头。

如果有向图满足对称关系, 那么将来回箭头合并为直线。

- 反对称关系

A relation R is antisymmetric if whenever $(a, b) \in R$ and a and b are distinct, then $(b, a) \notin R$. 可以有环, 但是不同的元素不能有双向关系, 请注意 (a,a) 是反对称的。

- 传递性

A binary relation is transitive if whenever $(a, b) \in R$ and $(b, c) \in R$, then $(a, c) \in R$.

- 等价关系

An equivalence relation is a relation that is reflexive, symmetric, transitive.

等价关系由包含很多 cluster(簇) 的无向图表示, 等价关系中的簇又被称为等价类。

簇: 等价关系可以划分出簇, 不同的簇不能有交集, 所有簇的并集应等于原集合。

等价类是一个集合，已知等价关系，我们用 $[a]$ 表示包含元素 a 的等价类关系， $[a] = \{b : (a, b) \in R\}$ 。

1.3.1 Theorem

Let R be an equivalence relation on a nonempty set A . Then the equivalence classes of R can constitute the partition of A .

根据划分和等价类的定义，我们需要证明由 $[a]$ 组成的等价类集合是非空，其中自己互斥并且所有子集的并集是原集合。这里我们使用数学中的反证法。

By reflexive, $(a, a) \in R$, $a \in [a]$, $a \in A$, all equivalence classes are nonempty.

To show that they are disjoint, consider any two distinct equivalence classes $[a]$ and $[b]$, and suppose that $[a] \cap [b] \neq \emptyset$. Thus there is an element c , $c \in [a]$ and $c \in [b]$. Hence in equivalence relation, $(a, c) \in R$ and $(c, b) \in R$, and $(a, b) \in R$ by transitivity; and since symmetric in R , $(b, a) \in R$. For any element $d \in [a]$, $(d, a) \in R$; by transitivity $(d, b) \in R$; hence $d \in [b]$. Hence $[a] \subseteq [b]$. Likewise $[b] \subseteq [a]$. Therefore $[a] = [b]$. But this contradicts the assumption that $[a]$ and $[b]$ are distinct. So $[a] \cap [b] = \emptyset$.

Each element a of A is in A by reflexive of R .

- partial order(偏序关系)
 1. Partial order is a relation that is reflexive, antisymmetric and transitive.
 2. 全序关系, a partial order $R \subseteq A * A$ is a total order, if for all $a, b \in A$, either (a, b) or $(b, a) \in R$.
 3. 全序关系需要任意两个元素都能够被比较。
- Path: A path in a binary relation $R \subseteq A * A$ is a sequence $(a_1, \dots, a_n)$.
- Cycle: The path $(a_1, \dots, a_n)$ is a cycle if the a_i 's are all distinct and also $(a_n, a_1) \in R$.

1.4 Finite and Infinite Sets 有限集和无限集

- 有限集的基本属性是 size，即集合包含元素的数量。
- equinumerous(等势). Two sets A and B are equinumerous if there is a bijection $f : A \rightarrow B$.

1. If a set is equinumerous with the natural number set $\{1, 2, \dots, n\}$ (n 是一个确定的数值); the set is finite set and the cardinality of the set is n. In summary, The cardinality of finite set is the number of the elements in it.

- infinite set 无限集

1. 如果一个集合不是有限集，那么就是无限集。
2. 如果一个无限集与自然数集 N 等势，即无限集中每一个数都能够与自然数集中每一个数对应，则称无限集为可数无限集。
3. Dovetailing(交织法) can prove that the union of a countably infinite collection of countably infinite sets is countably infinite. $A = \{a_0, a_1, \dots\}$, $B = \{b_0, b_1, \dots\}$, $C = \{c_0, c_1, \dots\}$, Then their union can be listed as $A \cup B \cup C = \{a_0, b_0, c_0, a_1, b_1, c_1, a_2, \dots\}$
4. N is natural number set, show $N \times N$ is countably infinite.(使用了康托尔配对函数)
在第 0 层，有一对 (0,0); 在第 1 层，有两对 (1,0)(0,1); 在第 2 层，有三对 (0,2)(1,1)(2,0); 在第 w-1 层，(0,w-1)(1,w-2)...(w-1,0)，从 0 到 w-1，一共有 w 对。所以在 w 层之前，我们一共有

$$1 + 2 + 3 + \dots + w = \frac{w(1+w)}{2} \quad (1)$$

$w = i + j, (i, j)$ 。在第 w 层，i 表示在这一层第 i 个被访问的。总序号 $m = \frac{w(1+w)}{2} + i$ 得到康托尔配对标准式子 $m = \frac{(i+j)(i+j+1)}{2} + i$ 因此我们可以将 $N \times N$ 与自然数集 N 进行双射。 $N \times N$ 是一个可数无限集。

1.5 Mathematics Proof Technique

- Mathematical induction

数学归纳法一般用来证明给定命题在自然数的整个或者部分范围是正确的。

数学归纳法的原理, 首先证明该命题在某个起始值是成立的, 然后证明从一个值到下一个值的过程也是成立的。当这两点都证明成立后, 反复使用这种方法, 就能推导出任何值的情况。

Example 1.5.1: Let us show that for any $n \geq 0$, $1 + 2 + \cdots + n = \frac{n^2+n}{2}$.

Basis Step. Let $n = 0$. Then the sum on the left is zero, since there is nothing to add. The expression on the right is also zero.

Induction Hypothesis. Assume that, for some $n \geq 0$, $1 + 2 + \cdots + m = \frac{m^2+m}{2}$ whenever $m \leq n$.

Induction Step.

$$\begin{aligned} 1 + 2 + \cdots + n + (n + 1) &= (1 + 2 + \cdots + n) + (n + 1) \\ &= \frac{n^2 + n}{2} + (n + 1) \quad (\text{by the induction hypothesis}) \\ &= \frac{n^2 + n + 2n + 2}{2} \\ &= \frac{(n + 1)^2 + (n + 1)}{2} \end{aligned}$$

图 1: Mathematical-induction

- Pigeonhole Principle(鸽巢原理) 若 A 和 B 是有限集, $|A| > |B|$, 则不存在从 A 到 B 一一对应的函数。

将 $n + 1$ 个苹果，放到 n 个抽屉中，那么有至少一个抽屉有两个（或以上）的苹果。

这个定理看起来比较显然，证明方法考虑反证法：

假如每个抽屉有最多 1 个苹果，那么最多有 $1 \times n$ 个苹果，而实际上有 $n + 1$ 个苹果，矛盾。

更加形式化的描述为：

- 将 $n + 1$ 个物体，划分为 n 组，那么有至少一组有两个（或以上）的物体。

推广

将 n 个物体，划分为 k 组，那么至少存在一个分组，含有大于或等于 $\left\lceil \frac{n}{k} \right\rceil$ 个物品。

推广的形式统一也可以用反证法证明：

若每个分组含有小于 $\left\lceil \frac{n}{k} \right\rceil$ 个物体，则对于总和 S 有

$$\begin{aligned} S &\leq \left(\left\lceil \frac{n}{k} \right\rceil - 1 \right) \times k \\ &= k \left\lceil \frac{n}{k} \right\rceil - k \\ &< k \left(\frac{n}{k} + 1 \right) - k \\ &= n \end{aligned}$$

故矛盾。

图 2: Pigeonhole-Principle1

• 定理 1.5.1: 设 R 是有限集 A 上的二元关系，且 $a, b \in A$ 。
若在 R 中存在从 a 到 b 的路径，则存在一条长度至多为 $|A|$ 的路径。

证明：假设 $((a_1, a_2, \dots, a_n))$ 是从 $a_1 = a$ 到 $a_n = b$ 的最短路径，即长度最小的路径，并且假设 $n > |A|$ 。

根据鸽巢原理，集合 A 中存在一个元素在路径上重复出现，例如 $a_i = a_j$ 、 $0 < i < j < n + 1$ 。但这样的话，

$(a_1, a_2, \dots, a_i, a_{j+1}, \dots, a_n)$ 就是从 a 到 b 的一条更短路径，这与我们假设的 $((a_1, a_2, \dots, a_n))$ 是从 a 到 b 的最短路径相矛盾。

图 3: Pigeonhole-Principle2

- Diagonalization Principle(对角化理论)

R 是集合 A 的二元关系, 设 D 是 R 的对角集 (Diagonalization set),
 $D = \{a : a \in A \text{ and } (a, a) \notin R\}$;

for each $a \in A$, define $R_a = \{b : b \in A \text{ and } (a, b) \in R\}$, R_a 是行集, 即所有与 a 有关系 R 的元素集合。

由此我们可以发现, 对于 diagonal set D , 如果 element a 存在 reflexive relation, 那我们就不将其加入 D 集合中, 若没有自反关系, 我们加入对角集, 即我们取主对角线的补集。 R_a 对应数组的每一行, 对于元素为 a 的行, 取 b (b 可以和 a 相等)。

- eg: The set 2^N is uncountable.

Supposing 2^N is countable infinite set, there is a way of enumerating all members of $2^N = \{R_0, R_1, R_2, \dots\}$, R_i is R_a in the statement of diagonalization principle. Consider an diagonal set $D = \{n \in N : n \notin R_n\}$. D is a set of natural number, D should be in R_n , $n = 1, 2, \dots$. But according to diagonalization principle, D can not appear in R_n , then it is a contradiction.

So, the set 2^N is uncountable.

1.6 Closures and Algorithm

- 自反传递闭包

1. Let $R \subseteq A^2$ be a directed graph defined on a set A . The **reflexive transitive closure** of R is the relation

$$R^* = \{(a, b) \mid a, b \in A \text{ and there is a path from } a \text{ to } b \text{ in } R\},$$

and R^* is reflexive. R^* 是包含 R 并且同时满足自反和传递的最小关系。

2. 算法

Initially $R^* := \emptyset$

for $i = 1, \dots, n$ do
 add (a_i, a_i) to R^*

for $i = 2, \dots, n$ do
 for each i -tuple $(b_1, \dots, b_i) \in A^i$ do
 if $(b_1, \dots, b_i)$ is a path in R then add (b_1, b_i) to R^* .

图 4: reflexive-transitive-closure

- 闭包算法定义

1. Let D be a set, let $n \geq 0$, and let $R \subseteq D^{n+1}$ be a $(n+1)$ -ary relation on D . Then a subset B of D is said to be **closed under** R if $b_{n+1} \in B$ whenever $b_1, \dots, b_n \in B$ and $(b_1, \dots, b_n, b_{n+1}) \in R$.
 R 是 D 的 $n+1$ 元关系，如果通过输入 B ，将 b_1 到 b_n 通过关系 R 输出得到 b_{n+1} ，那么 b_{n+1} 也属于 B 。
2. 闭包从一个较大的集合 D 和 D 的子集 A 开始，构建包含 A 且满足闭包性质的最小集合 B 。
3. (1) B contains A ; (2) B is the minimal set containing A and A and closed under $R_1, R_2, \dots, R_m$

1.7 Alphabets and Languages 字母表和语言

- **Alphabet**: Finite set of symbols (e.g., $\{0, 1\}$).
- **String**: Finite sequence of symbols from an alphabet.
- **Empty string**: String with no symbols, denoted e .
- **Length**: $|w|$ = number of symbols in string w ; $|e| = 0$.
- **Indexing**: $w(j)$ = symbol at position j in string w . 下标从 1 计算。
- String Operation

1. String concatenation, denoted by $x \circ y$ or xy . x is the prefix of w , y is the suffix of w . $w = x \circ y$ if and only if:

$$|w| = |x| + |y|,$$

$$w(j) = x(j) \text{ for all } j = 1, \dots, |x|,$$

$$w(|x| + j) = y(j) \text{ for all } j = 1, \dots, |y|.$$

2. For each string w and each natural number i , the string w^i is defined recursively as:

$$w^0 = \varepsilon,$$

$$w^{i+1} = w^i \circ w, \quad \text{for each } i \geq 0.$$

3. 字符串反转 reversal of string

The reversal of a string w , denoted by w^R . Details as follows: (1) If w is a string of length 0, then $w^R = w = e$. (2) If w is a string of length $n+1 > 0$, then $w = ua$ for some $a \in \Sigma$, and $w^R = au^R$.

Proof: For any strings w and x , $(wx)^R = x^R w^R$.

Basis Step. $|x| = 0$. Then $x = e$, and $(wx)^R = (we)^R = w^R = ew^R = e^R w^R = x^R w^R$.

Induction Hypothesis. If $|x| \leq n$, then $(wx)^R = x^R w^R$.

Induction Step. Let $|x| = n + 1$. Then $x = ua$ for some $u \in \Sigma^*$ and $a \in \Sigma$ such that $|u| = n$.

$$\begin{aligned} (wx)^R &= (w(ua))^R && \text{since } x = ua \\ &= ((wu)a)^R && \text{since concatenation is associative} \\ &= a(wu)^R && \text{by the definition of the reversal of } (wu)a \\ &= au^R w^R && \text{by the induction hypothesis} \\ &= (ua)^R w^R && \text{by the definition of the reversal of } ua \\ &= x^R w^R && \text{since } x = ua \end{aligned}$$

图 5: string-reverse

- infinite strings

1. 字母表 Σ 中字符组成的所有字符串包括空字符串在内的字符串记作 Σ^* .
2. language: Any set of strings over an alphabet Σ , or any subset of Σ^* .
3. 语言是一个集合, 包含的元素数量可以是无限的。我们使用式子来定义语言: $L = \{w \in \Sigma^*, w \text{ has property } P\}$.

4. Proof Σ^* is a countably infinite set?

这段话的核心是：证明“字母表 Σ 的所有字符串集合 Σ^* 是可数无限集”，方法是构造从自然数集 $\mathbb{N}$ 到 Σ^* 的双射（一一对应），本质是“按长度分组 + 字典序排序”的枚举法，通俗拆解如下：

第一步：明确核心目标

要证 Σ^* 可数，只需找到一种“无遗漏、不重复”的方式列出所有字符串——即构造双射 $f: \mathbb{N} \rightarrow \Sigma^*$ ，让每个自然数都对应唯一字符串，每个字符串也对应唯一自然数。

第二步：枚举规则拆解（2 条规则 + 例子，一看就懂）

先约定： Σ 是有限字母表（比如 $\{0,1\}$ ）， Σ^* 是所有字符串（含空字符串 ϵ ，长度为 0）。

规则 1：按“字符串长度”分组，短的在前

- 长度 $k=0$ （空字符串）：先列（只有 ϵ ）；
- 长度 $k=1$ ：再列所有 1 个字符的字符串；
- 长度 $k=2$ ：再列所有 2 个字符的字符串；
- ... 以此类推，永远先列完短字符串，再列长字符串。

规则 2：同长度字符串，按“字典序”排列

- 字典序：像查字典一样，按字母表预设的顺序（比如 $\Sigma=\{0,1\}$ 预设“0 在前，1 在后”）比较每个位置的字符；
- 同长度字符串有 n^k 个（ n 是 Σ 的字母个数， k 是长度）：比如 $\Sigma=\{0,1\}$ （ $n=2$ ）， $k=2$ 时，有 $2^2 = 4$ 个字符串（00,01,10,11），按字典序排列就是这个顺序。

图 6: countably-language-set

5. Operations on Languages

Languages can be combined using set operations to define new languages. 例如， $\bar{A}$ 表示 A 的补集，定义为 $\bar{A} = \Sigma^* - A$ 。

6. The Concatenation of Languages Let $L_1, L_2 \subseteq \Sigma^*$ be languages over alphabet Σ . The *concatenation* of L_1 and L_2 , denoted $L_1 L_2$ or $L_1 \circ L_2$, is defined as:

$$L_1 L_2 = \{w \in \Sigma^* : w = x \circ y, \text{ for some } x \in L_1, y \in L_2\}.$$

7. Kleene stare denoted L^* . L^* 是通过连接 L 中 0 个或多个字符串得到的所有字符串的集合。

1.8 Finite Representations of Languages 语言的有限表示

可数有限或者可数无限语言能够使用语言的有限表示进行表示，比如正则表达式，不可数无限语言不能使用有限表示进行表示。

- Example 1.8.1 Let $L = \{w \in \{0,1\}^* : w \text{ has two or three occurrences of } 1, \text{ the first and second of which are not consecutive}\}$
- This language can be described using only singleton sets and the symbols $\cup$, $\circ$ and $*$ as $\{0\}^* \circ \{1\} \circ \{0\}^* \circ \{0\} \circ \{1\} \circ \{0\}^* ((\{1\} \circ \{0\}^*) \cup \emptyset)$,
We may dispense with the braces and $\circ$ and write simply $L = 0^*10^*010^*(10^* \cup \emptyset)$

图 7: two-or-three-occurrences1

- regular expression
 1. **语义函数 L** : 设 a 是正则表达式, 则 $L(a)$ 表示 a 所描述的语言。
函数 L 从正则表达式映射到语言。
 2. **基础定义 (Base Cases)**:

$$L(\emptyset) = \emptyset, \quad L(a) = \{a\} \quad \text{对每个 } a \in \Sigma$$

其中 $\emptyset$ 表示空集 (不接受任何字符串), $\{a\}$ 表示只包含单个字符 a 的语言。

- (1) $\mathcal{L}(\emptyset) = \emptyset$, and $\mathcal{L}(a) = \{a\}$ for each $a \in \Sigma$.
- (2) If α and β are regular expressions, then $\mathcal{L}((\alpha\beta)) = \mathcal{L}(\alpha)\mathcal{L}(\beta)$.
- (3) If α and β are regular expressions, then $\mathcal{L}((\alpha\cup\beta)) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta)$.
- (4) If α is a regular expression, then $\mathcal{L}(\alpha^*) = \mathcal{L}(\alpha)^*$.

图 8: RegularExpression-BaseCases

3. **递归构造 (Recursive Construction)**: 若 α 和 β 是正则表达式, 则:

– 连接 (Concatenation):

$$L((\alpha\beta)) = L(\alpha) \circ L(\beta)$$

– 并 (Union):

$$L((\alpha \cup \beta)) = L(\alpha) \cup L(\beta)$$

– Kleene 星 (Kleene Star):

$$L(\alpha^*) = L(\alpha)^*$$

- (1) $\mathcal{L}(\emptyset) = \emptyset$, and $\mathcal{L}(a) = \{a\}$ for each $a \in \Sigma$.
- (2) If α and β are regular expressions, then $\mathcal{L}((\alpha\beta)) = \mathcal{L}(\alpha)\mathcal{L}(\beta)$.
- (3) If α and β are regular expressions, then $\mathcal{L}((\alpha \cup \beta)) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta)$.
- (4) If α is a regular expression, then $\mathcal{L}(\alpha^*) = \mathcal{L}(\alpha)^*$.

图 9: RegularExpression-operation

1.8.2: What is $\mathcal{L}(((a \cup b)^* a))$? We have the following.

$$\begin{aligned}
 \mathcal{L}(((a \cup b)^* a)) &= \mathcal{L}((a \cup b)^*) \mathcal{L}(a) \text{ by (2)} \\
 &= \mathcal{L}((a \cup b)^*) \{a\} \text{ by (1)} \\
 &= \mathcal{L}((a \cup b))^* \{a\} \text{ by (4)} \\
 &= (\mathcal{L}(a) \cup \mathcal{L}(b))^* \{a\} \text{ by (3)} \\
 &= (\{a\} \cup \{b\})^* \{a\} \text{ by (1) twice} \\
 &= \{a, b\}^* \{a\} \\
 &= \{w \in \{a, b\}^* : w \text{ ends with an } a\}
 \end{aligned}$$

图 10: RegularExpression-operation-eg

4. **正则语言 (Regular Languages)**: 字母表 Σ 上的正则语言类定义为:

$$\{L \subseteq \Sigma^* \mid L = L(a) \text{ 对某个正则表达式 } a\}$$

即: 正则语言是能用正则表达式描述的所有语言。

5. **闭包刻画 (Characterization via Closure)**: 正则语言类是集合

$$\{\{w\} \mid w \in \Sigma^*\} \cup \{\emptyset\}$$

在并、连接和 Kleene 星运算下的闭包。

6. 问题但是对于一些情况, 我们还是没法使用正则表达式进行表示。

- language generator

Language generator describe how a generic specimen in the language is produced

例如, 像 $(e \cup b \cup bb)(a \cup ab \cup abb)^*$ 这样的正则表达式可以被视为生成某一语言成员的一种方式:

要生成 L 中的一个成员, 首先写下空字符串、 b 或 bb ; 然后写下 a 、 ab 或 abb , 并可以进行任意次数 (包括零次) 的这种操作; 所有且仅有 L 的成员可以通过这种方式生成。

图 11: language-generator

- language recognition device

定义: 专门为某个语言 L 设计的算法, 用于回答形如“字符串 w 是否属于 L ?”的问题, 称为该语言的**语言识别装置**。

关键要求: 必须存在算法能确定任意给定字符串是否属于该语言。

Given a language L :

$$L = \{w \in \Sigma^* : w \text{ has property } P\},$$

there will be another problem:

For a given string w , you need to consider whether w belongs to language L : **there must be an algorithm for deciding whether a given string belongs to the language**. An algorithm that is specifically designed, for some language L , to answer questions of the form "Is string w a member of L ?" will be called a **language recognition device**.

给定一种语言 L :

$$L = \{w \in \Sigma^* : w \text{ 具有性质 } P\}, \text{ 则会}$$

出现另一个问题:

对于给定的字符串 w , 你需要考虑 w 是否属于语言 L : 必须存在一种算法来判定给定的字符串是否属于该语言。专门为某一语言 L 设计、用于回答“字符串 w 是 L 的成员吗?” 这类问题的算法将被称为语言识别装置。



1.8 Finite Representations of Languages

For example, recognize the language

$$L = \{w \in \{0,1\}^* : w \text{ does not have } 111 \text{ as a substring}\}.$$

A device for recognizing the language L by reading strings, a symbol at a time, from left to right, might operate like this: **Keep a count, which starts at zero and is set back to zero every time a 0 is encountered in the input; add one every time a 1 is encountered in the input; stop with a No answer if the count ever reaches three, and stop with a Yes answer if the whole string is read without the count reaching three.**



1.8 语言的有限表示

例如, 识别语言 $L = \{w \in \{0,1\}^* : w \text{ 不包含子串 } 111\}$ 。

一种通过从左到右逐符号读取字符串来识别语言 L 的设备, 其工作方式可能如下: 保持一个计数, 该计数从 0 开始, 每次在输入中遇到 0 时就重置为 0; 每次遇到 1 时就加 1; 如果计数达到 3, 就停止并给出“否”的答案; 如果读完整个字符串而计数未达到 3, 就停止并给出“是”的答案。

图 12: language-recognition-device